

Autonomous Software Agents Project

Dal Farra Christian
258243

Panciera Marco
257852

`christian.dalfarra@studenti.unitn.it` `marco.panciera@studenti.unitn.it`

Abstract

This report describes the development of an autonomous agent for the `Deliveroo.js` environment based on the Belief–Desire–Intention (BDI) architecture. The agent maintains structured beliefs about the map, parcels, opponents, and teammate state through event-driven updates and a dynamic obstacle overlay. It generates pickup, delivery, and exploration options scored by a decay-aware utility function combining reward, distance, and opponent proximity. The selected intention is executed via a preemptive scheduler and modular plans, using either A pathfinding or an optional PDDL planner. In multi-agent mode, lightweight message-based coordination enables belief sharing, conflict avoidance, and recovery from blocking situations*

1 Introduction

The project focuses on the `Deliveroo.js` environment, a parcel pickup-and-delivery task set in a discrete two-dimensional grid. Parcels appear on the map at random spawn locations, gradually lose value as time passes, and yield points when successfully delivered to a delivery tile. Agents operate in a competitive and partially observable environment: they must choose which parcels to pursue, plan paths across the grid, avoid conflicts with other agents, and decide when to deliver in order to maximize their score.

To address this problem we adopt the Belief–Desire–Intention (BDI) paradigm, a widely used model for autonomous agents. The BDI framework provides a natural separation of concerns:

- **Beliefs:** the information the agent maintains about the world, including the static map, its own state, perceived parcels, and other agents.
- **Desires/Options:** the set of possible goals or tasks the agent could attempt, generated from its beliefs.
- **Intentions:** the selected desires that the agent commits to achieving, managed through an intention queue and subject to revision.
- **Plans:** concrete strategies for achieving intentions, such as moving to a location via A* pathfinding, picking up a parcel, or putting one down.

This approach is well suited for `Deliveroo.js` because it allows the agent to reason continuously under uncertainty, to adapt to a dynamic environment, and to balance between opportunistic parcel collection and efficient delivery. Moreover, the modular structure of BDI supports extension to multi-agent coordination and makes the decision process transparent and debuggable.

1.1 Game Environment

The world is represented as a grid where each cell corresponds to a tile. Tiles are encoded numerically and determine both traversability and function:

- **0: Wall** — impassable, parcels cannot spawn here.
- **1: Spawn tile** — walkable; parcels may appear at these locations.
- **2: Delivery tile** — walkable; agents can deliver parcels here to gain points.
- **3: Normal walkable tile** — traversable but without special functions.

Agents can move in the four cardinal directions (up, down, left, right). Each move takes a fixed duration defined by the environment configuration. Agents perceive parcels and other agents within a limited observation distance, which constrains their knowledge and makes belief management central to good performance. Available actions include *pickup* (collecting a parcel when on the same tile) and *putdown* (delivering a carried parcel when on a delivery tile).

1.2 Agent Objectives

The objectives of the agent can be summarized as follows:

1. **Acquire valuable parcels:** identify and pursue parcels that will retain a positive reward by the time the agent arrives.
2. **Deliver efficiently:** deposit parcels at delivery tiles when it is advantageous, either because the carried reward is significant or because no better pickup opportunities exist.
3. **Handle conflicts and obstacles:** adapt to dynamic conditions such as enemy agents blocking a path or racing for the same parcel.
4. **Coordinate in multi-agent settings:** exchange minimal but useful information (such as parcel locations and intentions) with a teammate in order to avoid redundant actions and improve collective efficiency.

2 System Architecture

The agent is organized around the Belief–Desire–Intention (BDI) paradigm. Each module reflects one stage of the reasoning cycle, and information flows through structured data classes and Deliveroo.js callbacks. This modularity improves maintainability and allows independent refinement of each component.

2.1 Modules Overview

- **MapData.js:** maintains the static map and a dynamic copy (*utilityMap*) where enemies are treated as obstacles.
- **EnvData.js:** stores configuration constants such as movement duration, decay frequency, and parcel reward statistics.
- **AgentData.js:** tracks agent identity, position, known parcels, carried parcels, available options, current intention and mate intention.
- **belief.js:** connects to the Deliveroo.js API and updates the internal belief state via event callbacks.

- **options.js**: generates candidate options from the belief state and scores them with a utility function.
- **intention.js**: implements the `Intention` and `IntentionRevision` classes, which manage intention queues and execution.
- **plans.js**: provides concrete plan classes for navigation, picking up parcels, and delivering them.
- **coordination.js**: defines message formats and functions for inter-agent communication in multi-agent mode.

3 Belief Modeling

Beliefs are maintained across three primary data structures: `MapData`, `AgentData`, and `EnvData`. They capture static information, dynamic observations, and configuration parameters, respectively.

3.1 MapData

The `MapData` class represents the map:

- **map**: static matrix of tile types (0 = wall, 1 = spawn, 2 = delivery, 3 = walkable).
- **utilityMap**: dynamic copy updated at runtime, where enemy positions are marked as non-walkable.
- **deliverCoordinates**: coordinates of all delivery tiles for the put down of parcels.
- **spawnCoordinates**: coordinates of all tiles where can spawn parcels.
- **walkableCoordinates**: coordinates of walkable tile without particular function.
- **width, height**: map dimensions.

A method `updateTileValue(x, y, value)` allows modifying specific cells of the `utilityMap` to reflect dynamic obstacles.

3.2 AgentData

The `AgentData` class tracks the agent's state:

- **Identity and position**: `name`, `id`, `pos`, and current `score`.
- **Parcel knowledge**: `parcels` (all observed or communicated parcels that are not carried) and `parcelsCarried` (parcels currently held by the agent).
- **Options**: `options` (candidate actions) and `best_option` (highest-scoring option).
- **Enemies**: list of perceived enemy agents, with timestamps.
- **Intentions**: `currentIntention`, `mateId`, `mateIntention`, `matePosition`, and `myIntentions` (object that manage the queue of intentions that i want to execute).

3.3 EnvData

The `EnvData` class centralizes environment parameters:

- `parcel_reward_avg`, `parcel_reward_variance`: statistical properties of parcel rewards.
- `parcel_observation_distance`, `agents_observation_distance`: perception radius, distance from which the agent can see parcels and agents.
- `movement_duration`: duration of a single move.
- `decay_frequency`: effective decay rate for parcels, derived from movement duration and decay interval.
- `clock`: base interval for triggering the option loop.

3.4 Belief Updates via Callbacks

Belief updates are event-driven and managed in the `belief.js` script:

- **onYou**: initializes identity values on the first call, then updates the position and say to the mate the actual position for the multi-agent part.
- **onParcelsSensing**: integrates visible parcels on the ground with timestamps, decays unseen parcels proportionally to elapsed time, and discards them if reward ≤ 0 . Carried parcels are tracked separately. If in multi-agent mode, newly perceived parcels are broadcast to the teammate.
- **onAgentsSensing**: refreshes enemy positions, removes those unseen for more than $10 \times$ `movement_duration`, and marks visible enemies as walls in the `utilityMap`.
- **onMsg**: processes teammate messages. Supported types include:
 - `say_intention`: update the teammate's current intention,
 - `say_agents`: placeholder, currently unused.
 - `say_position`: used in multi-agent to communicate the current position to our mate.
 - `ask_pick_up`: in the case we have to ask the mate a parcel that we have dropped.

4 Option Generation

The generation of options is the bridge between beliefs and intentions. At each iteration of the reasoning loop, the agent evaluates the current belief state, produces a set of feasible options, assigns a utility score to each, and selects the best one to commit to as an intention. This process is handled primarily in `options.js`.

4.1 Option Types

Three main categories of options are generated:

- **Pickups** (`go_pick_up`): Options to move towards a parcel and collect it. A pickup option is generated only if:
 - The parcel is not already carried by another agent.
 - The parcel's tile is not blocked in the `utilityMap`.
 - The parcel's expected reward at pickup remains positive after accounting for decay.

- **Deliveries** (`go_put_down`): Options to deliver carried parcels at the nearest delivery tile. These are generated when either:
 - The agent holds parcels and no viable pickup options exist, or
 - The expected delivered value exceeds $1.5\times$ the average parcel reward.
- **Exploration** (`go_to`): If no pickups or deliveries are available, the agent generates a random walk to a spawn tile or a walkable tile if no spawn tile are reachable. This ensures continuous exploration of the environment and avoids deadlock.

4.2 Utility Function

Each pickup option is scored using a composite utility function defined in `pickUpUtility(parcel)`. The utility combines several factors:

$$U(parcel) = U_{time} + U_{reward} + U_{enemy} + U_{cluster} \quad (1)$$

where:

- U_{time} : recency score, higher for parcels observed recently (maximum 12, minimum 2).
- U_{reward} : expected reward at pickup time, accounting for decay along the path.
- U_{enemy} : proximity comparison with the nearest enemy, rewarding parcels the agent can reach before others.
- $U_{cluster}$: bonus proportional to the number of nearby parcels (10 per adjacent parcel).

Formally, let r be the current reward of the parcel, d_{me} the distance from the agent, d_{enemy} the distance of the nearest enemy, and δ the decay frequency. Then:

$$U(parcel) = f_{time} + (r - d_{me} \cdot \delta) + \mathbf{1}[d_{enemy} > d_{me}] \cdot 10 + (n_{close} \cdot 10)$$

where n_{close} counts adjacent parcels and f_{time} is a time-based constant. Only parcels with $U(parcel) > 20$ are considered viable.

For deliveries, a utility $10 \times n_parcels_carried$, . Exploration options receive a baseline utility of 0.

5 Intention Management

Once options are generated and scored, they are pushed into the intention management system, defined in `intention.js`. The agent employs an intention revision loop (`IntentionRevision`) that maintains a queue of pending intentions.

5.1 Intention Queue

- Each option is wrapped as an `Intention` object, storing its predicate and associated plan.
- Duplicate intentions (same type and goal) are not added to the queue.
- The queue is sorted by utility, ensuring higher-value intentions are pursued first.

5.2 Execution Loop

The `loop()` method of `IntentionRevision` runs continuously:

1. Take the top-ranked intention.
2. Attempt to achieve it by matching its type with applicable plans (e.g. `PickUp`, `PutDown`, `AStarGoTo`).
3. If the plan succeeds, the intention is removed from the queue and the `optionsLoop()` is triggered again.
4. If it fails, the intention is discarded and the agent re-evaluates options.

6 Plan Execution

Once an intention has been selected, it is achieved by executing an appropriate plan. Plans implement concrete strategies to transform high-level goals (e.g. *go-pick-up*) into sequences of primitive actions supported by Deliveroo.js (`move`, `pickup`, `putdown`). The available plans are defined in `plans.js`.

6.1 A* Pathfinding

Navigation relies on an A* search implementation, defined in `utils.js` and used by the `GoTo` plan. Paths are computed on the dynamic map (`utilityMap`), where enemy positions are treated as temporary walls, to avoid the enemies.

The plan proceeds as follows:

1. Given a goal coordinate, compute a path using `findAStar`.
2. Follow the path step by step, issuing `emitMove(action)` for each move.
3. If the path becomes invalid (e.g. due to an enemy blocking a tile), recompute a new path from the current position.
4. If no path exists, the plan fails and the intention is aborted.

Additional behaviors are integrated into the navigation:

- *Random walk guard*: if a navigation plan with utility = 0 is running, it may terminate early if better pickup options appear.
- *Pickup check during delivery*: while delivering, if nearby valuable parcels are detected, the plan may temporarily adopt a sub-intention to collect them.
- *Blocked delivery tiles*: if the delivery tile is occupied by an enemy, the plan switches to an alternative delivery location.

This design ensures robustness against dynamic obstacles and opportunistic adaptation to newly perceived parcels, solving problems encountered during the first challenge, where the movement of the enemies were bad managed so the `GoTo` plan fails most of times.

6.2 PickUp and PutDown Plans

Two additional plans specialize in parcel interaction:

- **PickUp:** This plan first generates a sub-intention `go_to` towards the parcel location using `AStarGoTo`. Once reached, it executes `emitPickup()`. If successful, the parcel is added to `parcelsCarried`. If the parcel is no longer available or blocked, the plan fails.
- **PutDown:** This plan generates a sub-intention `go_to` towards the nearest delivery tile. Upon arrival, it executes `emitPutdown()`. If successful, all carried parcels are removed from memory and points are scored. If the delivery tile becomes blocked and no alternative is found, the plan fails.

7 Planning with PDDL

In addition to the default A*-based reactive navigation, the agent can use a PDDL-based planning backend that generates a symbolic sequence of actions (move, pick-up, put-down) executed through the Deliveroo API. The backend is selected at startup (see Section ??).

7.1 PDDL Domain Model

The domain (`domain.pddl`) defines:

- adjacency relations between tiles (`up/down/left/right`),
- agent position (`at`),
- parcel location (`parcel_at`),
- carrying and delivered status (`carrying, delivered`),
- delivery tiles (`delivery`).

Movement updates the `at` predicate, pickup transfers a parcel from `parcel_at` to `carrying`, and put-down asserts `delivered` on delivery tiles. The domain abstracts from time, decay, and multi-parcel constraints, which are handled at the BDI level.

7.2 PDDL Problem Compilation

The method `createPddlProblem(predicate)` converts the current belief state into a PDDL problem.

Each walkable tile in the dynamic `utilityMap` becomes an object `tile_x_y`. Adjacency predicates are generated for neighboring walkable tiles. The initial state includes the agent position, delivery tiles, and—depending on the predicate—either the target parcel location or the currently carried parcel.

The goal depends on the intention:

- `go_to`: reach a target tile,
- `go_pick_up`: achieve `carrying(parcelId)`,
- `go_put_down`: achieve `delivered(parcelId)`.

7.3 Action Bindings and Execution

Symbolic actions are bound to real API calls through `createPddlActions(checkStopped)`, which creates interruptible `PddlAction` objects. Each action checks whether the intention has been stopped before execution, ensuring compatibility with BDI preemption.

Plans are executed by `PddlGoTo`, `PddlPickUp`, and `PddlPutDown`. Each plan builds a `PddlProblem`, calls `onlineSolver`, and executes the resulting plan via `PddlExecutor`. Navigation plans are retried if the solver returns no plan or the goal is not reached. Pickup and put-down actions are validated against current beliefs to detect inconsistencies (e.g., stolen parcels), triggering replanning if necessary.

7.4 Integration

The planner is selected at startup via `configurePlans(usePddl)`:

- `false`: classical A*-based plans,
- `true`: PDDL-based plans.

The BDI layer remains unchanged, as intentions always use the same predicates (`go.to`, `go.pick.up`, `go.put.down`), making the planning backend interchangeable.

8 Multi-Agent Coordination

The project supports both single-agent and two-agent execution. In multi-agent mode, two independent agent processes are spawned and connected to the same team via the Deliveroo messaging primitives.

8.1 Message Protocol

Coordination uses two channels:

- `emitSay`: asynchronous broadcast-like messages
- `emitAsk`: request-style message used for urgent interventions

Table 1: Implemented message types and their role in the BDI cycle.

Type	Semantics / BDI effect
<code>say_position</code>	Shares teammate position; used to treat teammate as dynamic obstacle and detect blocking.
<code>say_agents</code>	Shares perceived enemies; merged into beliefs and written into <code>utilityMap</code> as obstacles.
<code>say_intention</code>	Shares current intention predicate; used to prevent duplicated/conflicting targets.
<code>ask_pick.up</code>	Urgent request: receiver injects a forced high-utility pickup intention.

8.2 Belief Sharing and Fusion

The receiver merges teammate information in `belief.js`:

- enemy beliefs are augmented (if new) and their tiles are set to non-walkable in `utilityMap`
- teammate position is stored in `matePosition`

- teammate intention is stored as an `Intention` object (`mateIntention`)

This design keeps coordination lightweight: there is no global shared world state, only incremental belief synchronization.

8.3 Conflict Avoidance in Intention Selection

When pushing a new intention, the scheduler (`IntentionRevision.push`) rejects options that match:

- the current executing intention (avoid redundant re-adding)
- the teammate announced intention (avoid duplicated targets)

The predicate equality test is implemented by `checkOption` which compares `type` and goal coordinates (`x,y`). This acts as a simple distributed task allocation heuristic: “first agent to commit keeps the target.”

8.4 Cooperative Handover under Blocking

A specific cooperative behavior is implemented inside navigation execution (`GoTo.execute`) when the next planned tile is occupied by the teammate, and the agent is executing a delivery chain (parent predicate is `go_put_down`), and replanning cannot bypass the blockage, in that case, the agent:

1. puts down carried parcels on its current tile,
2. moves one step away to clear the corridor (if possible),
3. sends `ask_pick_up` to the teammate with the drop location,
4. waits for a short window, then stops its current intention and regenerates options.

On the receiver side, `ask_pick_up` triggers a forced pickup intention with very large utility, ensuring preemption of low-utility activities. This is effectively a *reactive joint-plan fragment*: it does not require explicit joint intention reasoning but still achieves cooperative recovery from deadlocks.

9 Results

In this section, we analyze the two in-class challenges, which provided valuable insights into the agent’s behavior in competitive environments with real opponents, as well as the numerical results obtained on different game maps.

In the first challenge, involving a single agent, we observed that parcel selection was suboptimal. To address this issue, we introduced a structured utility function (see Section 4.2) to improve decision-making during pickup. Another problem concerned delivery: the agent often continued searching for new parcels instead of performing put-down actions, due to thresholds that were set too high.

In the second challenge, conducted with two agents, we learned how to manage duplicated intentions. At times, both agents pursued the same goal, leading to conflicts and blocking situations. An unexpected behavior emerged on one map, where an agent blocked opponents from delivering parcels. After the challenge, we improved coordination by introducing explicit intention sharing.

Finally, we report numerical results obtained on three different maps, each run for three minutes. In the single-agent setup, we compare the classical (A*-based) agent with the PDDL-based

agent. In the multi-agent setup, only the classical planning approach was evaluated, the final score is the sum of the two agents.

Table 2: Single-agent results : A* vs PDDL.

Map	A* score	PDDL score	Δ (PDDL – A*)
24c1_1	945	306	-639
24c1_2	876	218	-658
24c1_3 3	732	249	-483

The results shows that the A* agent performs better then the PDDL, looking at the execution the PDDL agent is very slow due to request to server.

Table 3: Multi-agent results.

Map	Agent 1 score	Agent 2 score	Total score
24c1_9	1038	783	1821
24c2_5	609	586	1195
25c2_5	870	923	1793

10 Conclusion and Future Work

The final agent combines a BDI deliberation layer (belief update + option generation + intention scheduling) with two interchangeable planning backends (A* vs PDDL) and a minimal-yet-effective multi-agent protocol.

Future improvements suggested by the current implementation:

- **More expressive PDDL:** include time/decay or costs to align symbolic plans with reward dynamics, and the PDDL for the collaboration of the two agents.
- **Planning efficiency:** reduce replanning overhead and avoid repeated solve calls under frequent changes.
- **Better cooperation:** the agents should plan together for put down and pickups in order to reduce the movements and maximize the score.
- **Deep learning:** in the project the thresholds are fixed, instead the agent can learn those values based on the state, or better also learn a policy for the actions to take on experiences during the execution.